Box Transpose: In-Place N-Dimensional Tensor Permutation on GPUs

Kaveen Jayamanna Alexis Newman
Kennesaw State University
CS 8045 Advanced Algorithms

Abstract

Changing the order of a tensor's axes in PyTorch is free. Fixing the layout afterwards is not. `.contiguous()` makes a full copy, so peak memory doubles. On a GPU that is almost full, the job fails there.

We present *box transpose*. It fixes the layout in place. A shortest-path search breaks the permutation into swaps of neighbouring groups of axes. Each swap is a batched matrix transpose, which we carry out by following its cycles. The only extra memory is the list of where each cycle starts.

A copy has to write every value, even the ones already in the right place. We skip them. So box transpose can move less than the whole tensor, and a copy never can. How much it moves follows from the shape in one line of arithmetic. That predicts the speedup over `.contiguous()` before any data is touched, with no constant measured on a GPU. The prediction lands within about 2.4% of the measured time.

We tested 87 randomly drawn shapes at ranks 3 to 10, each 1,024 MB, on a NVIDIA GeForce RTX 2070. Box transpose needed 144 bytes of extra memory in the median case. Against three published in-place methods it was faster on 87, 87 and 79 of 87 cases, by 7.9× to 9.5× in the median. When the permutation leaves a large group of axes at the back alone, a check on the shape picks out the cases where we beat the copy itself. We won all 21 of them, 1.73× faster in the median. When that group is too small, we are about 30× slower than the copy, and the same check says so in advance.

1 The Problem

1.1 Changing the order of axes is free

Deep learning code changes the order of a tensor's dimensions often. In PyTorch, `permute` and `transpose` do this [7, 8]. They are free. They change a label. They do not move any data.

1.2 Fixing the layout is not free

After that, the data is in the wrong order in memory. Some operations will not run on it. Others run, but they use a slow path. They do not warn you.

The normal fix is `.contiguous()` [6]. It makes a copy. It asks for a new tensor. It writes every value into the right place. Then it throws the old tensor away.

1.3 The copy is the problem

While the copy runs, both tensors are in memory. So a tensor of B bytes needs at least $2B$ bytes. The old one must stay until it has been read. The new one must exist to be written into. This is what a copy means. You cannot avoid it by changing the loop.

If the machine is almost full, the job fails here. A 2026 PyTorch design document says non-contiguous tensors now appear all over modern training [12]. It says some communication routines will not take them. It says the usual fixes either make a full copy or need a buffer as big as the data.

There is a second problem. The copy is a large block of memory. It only lives for a short time. It appears in the middle of work that already uses a lot of memory. Earlier work shows this pattern causes fragmentation [10]. Memory is free, but it is in small pieces. The next request needs one big piece and fails.

1.4 What we do

We fix the layout in place. We never ask for a second tensor. We break the reordering into small swaps. Each swap exchanges two groups of axes that sit next to each other. We store only a few hundred bytes while we work.

Doing this in place is not new. Section 5 lists the earlier work. Two things here are new. In more than two dimensions, this can do *less work than a copy*. And we can tell which cases those are by looking at the shape, before we touch any data.

2 How It Works

2.1 Swapping boxes

A *box* is one or more axes that sit next to each other. We treat a box as one unit. Their sizes multiply together, so a box acts like a single axis of that size.

A *swap* exchanges two boxes that sit next to each other. Any reordering of axes can be done as a list of swaps.

Here is why that helps. Cut the axis list into four parts: a front part, a left box, a right box, and a back part. Call their sizes D_{pre}, *rows*, *cols* and D_{post}. Swapping the two middle boxes is the same as transposing a $rows \times cols$ matrix. We do it once for each of the D_{pre} front positions. Each cell of that matrix is a run of D_{post} values that sit next to each other.

Figure 1 shows this on a small shape. This idea is not ours. It is the $\rho(i, j, k)$ operation from ITTPD [2]. Section 5 says more. What is ours is how we run a swap, and how we choose which swaps to make.

2.2 Doing a swap with no spare room

You cannot move a value straight to where it belongs. Something else is already there. So you pick a value up. You put it where it goes. Now you are holding whatever was in that spot. You keep going. In the end you come back to where you started.

That loop is called a *cycle*. Walking it is called cycle following [3, 5]. The only thing you store is the one value in your hand. On a GPU that is one register per thread.

In two dimensions this does not spread well over a GPU. You can only run as many cycles at once as there are cycles. Catanzaro et al. [1] say this plainly. A shape with nine cycles gets nine-way parallelism, even on a machine with hundreds of processors.

With more dimensions that limit goes away. We repeat the swap D_{pre} times. Each value we move is D_{post} values wide. So the work splits into $D_{pre} \times cycles \times D_{post}$ pieces that can run at the same time. Those two extra numbers come from the axes around the swap. A two-dimensional method does not have them.

2.3 Skipping cells that are already correct

Some cells are already in the right place. A transpose never moves the diagonal of a square matrix.

We skip those cells. A copy cannot. `.contiguous()` is building a new tensor. It must write every position, even the ones that were already correct.

There is a formula for how many cells stay put. The transpose sends position p to $(p \bmod cols)\, rows + \lfloor p/cols \rfloor$. In the middle of the array this is the same as multiplying by *cols* modulo $M = rows \cdot cols - 1$. A cell stays put when $p(cols - 1) \equiv 0 \pmod{M}$. That has $\gcd(rows - 1, cols - 1)$ answers. The last position also stays put. So the share of the tensor a swap touches is

$$moved = 1 - \frac{\gcd(rows - 1, cols - 1) + 1}{rows \cdot cols}. \tag{1}$$

That is our whole cost model. A copy reads every value and writes every value. It always moves $2N$ values' worth of traffic. In the cases where our method is worth using, both methods move data at the same speed. So the speed cancels out. Only the amount is left. That makes Equation (1) our predicted time as a share of the copy's time. No number in it was measured on a GPU. If it is below one, we do less work than the copy.

Figure 2 draws this for two blocks. Now the main point. Any method that builds output positions must go over the tensor at least once. It has to write every position. Ours does not. Swapping a 2×2 block moves 0.50 of the tensor. No single swap can do better. The reason is that $\gcd(rows - 1, cols - 1)$ is at most $\min(rows, cols) - 1$, so the skipped share is at most $1/\max(rows, cols)$.

Two things follow. Only a method that skips can do part of a pass. And two swaps can never add up to less than one pass. So *beating* `.contiguous()` *and using a single swap are the same thing*.

2.4 Choosing which swaps to make

There are many lists of swaps that reach the same order. They do not cost the same.

We build a graph. Each way of ordering the axes is a point. Each legal swap is a line between two points. The weight on that line is how many bytes the swap moves. We know that number exactly from Equation (1). Then we run Dijkstra's algorithm to find the cheapest route.

Figure 3 shows the start of that search on a small shape. We do not remove any lines. We do not apply any cutoff to them. We only reject a finished route. If the cheapest route needs more than 6 swaps, we return nothing. The caller then uses `.contiguous()`. Every swap rewrites the whole buffer, so a long route moves several tensors' worth of data.

The search does not make us faster than a copy. Section 2.3 shows that needs a single swap, and you do not need a search to find one. The search matters for the longer routes. Nothing beats a copy there, but the search keeps us ahead of the other in-place methods.

2.5 Planning happens once

There are three phases, and only one of them runs often.

Planning builds the graph and runs the search. It touches no data at all. It only needs the shape and the permutation, so it runs on the CPU.

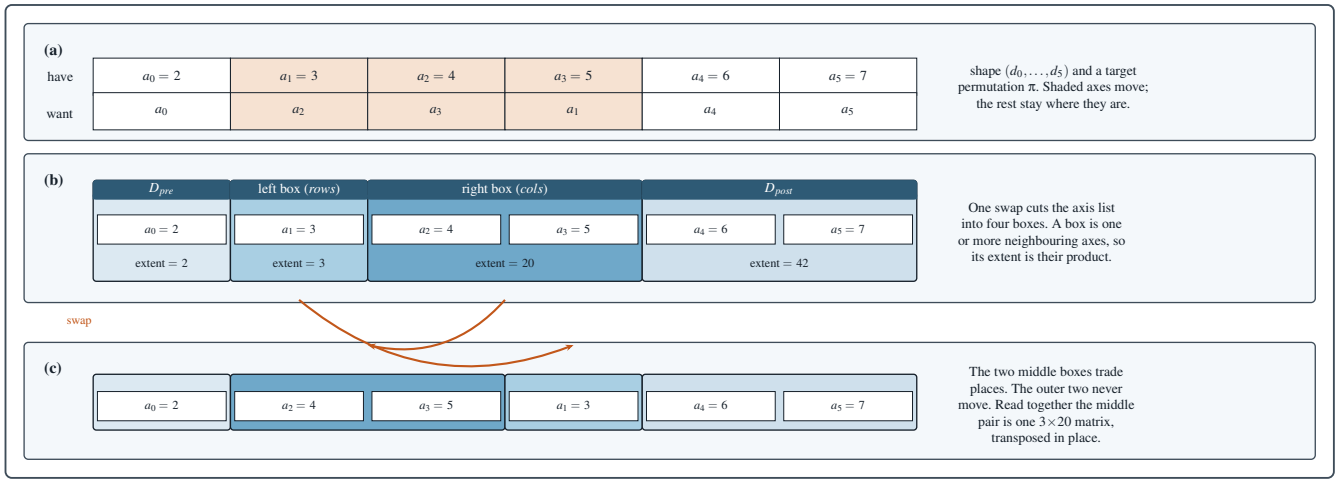


Figure 1: **How one swap cuts a shape into four boxes.** (a) The axes we have, and the order we want; shaded axes are the ones the permutation moves. (b) The four boxes a single swap cuts them into. A box is one or more neighbouring axes, so its extent is their product — here the right box covers two axes. (c) After the swap: the two middle boxes have traded places and the outer two have not moved. Read together, the middle pair is one $rows \times cols$ matrix, and each of its cells is a run of D_{post} values that sit next to each other in memory.

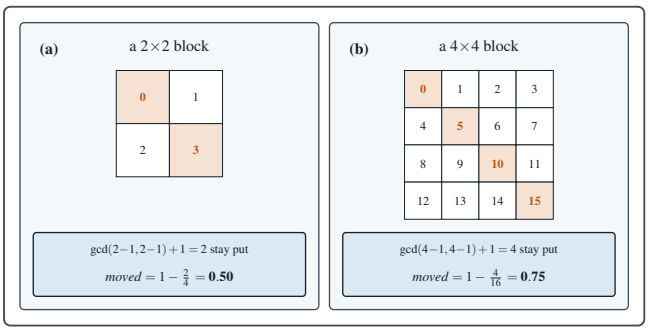


Figure 2: **Where an edge's weight comes from.** Each cell carries its flat position. Shaded cells are the ones a transpose maps to themselves — they are already in the right place. We skip them; a copy has to write them anyway. The smaller the block, the larger the share that stays put, so a 2×2 block is the best any single swap can do.

Preparing uploads what the kernel needs to the GPU. *Applying* runs the swaps on the real tensor. That last one is the only phase whose cost depends on how big the tensor is.

Planning is not free. For a single swap it returns at once, because the search stops as soon as it reaches the target. For a hard permutation at rank 8 it took several seconds (Section 4).

That would be a problem if we paid it every call. We do not. A plan depends only on the shape and the permutation, and neither changes from one call to the next. So we work it out once and keep it. The same tensor shape appears again and again in a training loop, so the first call pays for the plan and every call after it pays nothing.

This is why planning cost does not appear in any of our timings. We time applying, which is what runs in the loop. We say so here so that nobody reads the numbers in Section 3 as though the search ran inside them.

2.6 Nothing is tuned

An older version of this work used three numbers measured on one GPU. There was a throughput knee, a routing cutoff, and an average bandwidth. All three are gone. The bandwidth cancelled out. The knee became a question about the shape instead of a measurement.

We can check that the model knows where it works. Inside the region we claim, Equation (1) predicts the measured ratio to within 2.4%. Outside it, the error is about 64%. That is why the region exists.

3 Results

3.1 How we tested

We did not choose shapes by hand. Take a tensor with $N = \prod_{i=1}^{m} p_i^{a_i}$ values and rank r. Every valid shape comes from splitting each prime's exponent across the r dimensions, so that $\sum_{j=1}^{r} b_{ij} = a_i$:

$$s = \left(\prod_{i=1}^{m} p_i^{b_{i1}}, \prod_{i=1}^{m} p_i^{b_{i2}}, \ldots, \prod_{i=1}^{m} p_i^{b_{ir}} \right). \tag{2}$$

We draw from that whole set. So nobody picked the cases. We drop shapes with an axis of size one. Such an axis can move anywhere without touching a byte.

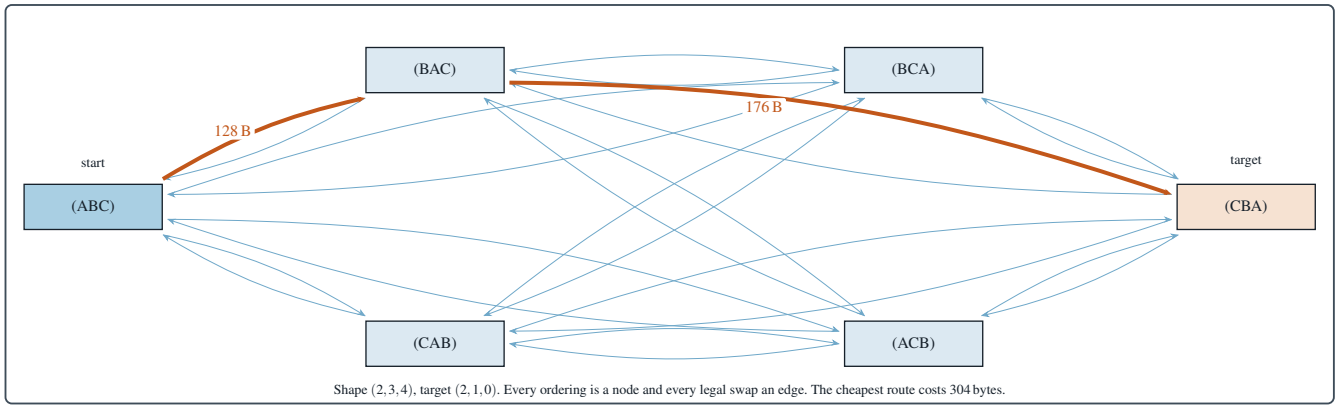


Figure 3: **The graph the planner searches.** Each node is one way of ordering the axes. Each arrow is a legal swap. The cheapest route is drawn in orange with the bytes each of its swaps moves. The weights are not equal, so the cheapest route is not simply the shortest one. This is the same example, and the same layout, as the step-by-step animation in `docs/dijkstra-animation.html`; the generator checks its cost model against the planner's, so the two cannot drift apart.

The main test has 87 cases at ranks 3 to 10. Every tensor has 268,435,456 float32 values, which is 1,024 MB. We ran them on a NVIDIA GeForce RTX 2070. Each shape is paired with a permutation that leaves a group of axes at the back alone. That is what our method is built for.

We ran a second test with 15 permutations picked at random. That covers what our method is not built for. Section 4 reports it.

We compare against PyTorch `.contiguous()`. We also compare against our rebuilds of the three published in-place methods. Section 5 lists them. We time every method with CUDA events after one warmup run. We check every answer against `permute(*perm).contiguous()` before it counts.

Our method finished and gave the right answer on every case in both tests. It turned nothing away.

3.2 Against everyone

Table 1 shows all 87 cases. We used no routing here. We ran our method even on the cases our own rule says to skip.

In the middle of the range we are 7.9× faster than ITTPD, 8.1× faster than Gomez-Luna, and 9.5× faster than Gustavson. Case by case, we are faster on 87 of 87, on 87 of 87, and on 79 of 87.

The memory column matters too. `.contiguous()` needs a second tensor every time. We need 144 bytes. Gustavson needs 48 MB. ITTPD needs none. That is real, not a mistake. On these shapes its blocks fit in fast on-chip memory, so it never asks for global memory. Against ITTPD we win on speed, not on memory. We do not claim otherwise.

3.3 Three cases

Table 2 splits both tests by the two numbers from Section 2. We work out both from the shape before touching any data.

The back group is large and the swap is one small square block. Here we are faster than everyone, including the copy. We won 22 of 22 cases, at 1.73× in the middle of the range. In the main test alone, our rule fires on 24% of shapes and wins 21 of 21. That is 1.73× in the middle and 2.05× at best.

The back group is large but the swap does not qualify. Here we lose to the copy. We are still the best in-place choice, on 69 cases.

The back group is too small. Here we are the worst. Section 4 covers it.

3.4 The prediction was right

We did not fit the boundary to the results. Equation (1) says a swap of a 2×2 block moves 0.50 of the tensor. So it should take 0.50 of the copy's time. From rank 3 to rank 10, the measured times land on that value. The error inside the region is about 2.4%.

The speedup comes from arithmetic, not from tuning. It does not depend on how well we wrote the kernel. It does not depend on which GPU we used. It depends only on how much of the tensor was already in the right place.

4 Where It Falls Down

4.1 A small back group means no speed

Some permutations move the innermost axis. Then every way of breaking the job up has a swap with nothing behind it. For that swap, $D_{post} = 1$.

Now each value we move is a single number on its own. The memory system reads a whole cache line and uses four bytes of it. There is no way to group the work.

Our second test measures this. We used 15 permutations picked at random. In 4 of them a back group survived. We

Table 1: **Every method, every case, no routing.** 87 shape and permutation pairs at ranks 3–10, 1,024 MB per tensor, on a NVIDIA GeForce RTX 2070. We run our method on all of them, including the ones our own rule would turn away.

Method	Median	Median aux	Peak aux	Box wins
PyTorch `.contiguous()`	7.70	1,024 MB	1,024 MB	32/87
Box transpose (ours)	**10.21**	**144 B**	**10.6 KB**	–
ITTPD [2]	80.49	0	0	87/87
Gomez-Luna [4]	82.91	16 B	512 B	87/87
Gustavson [5]	97.39	48 MB	128 MB	79/87

Middle-of-the-range values over every case, with no routing: we run our method on all of them, including the ones our own rule would turn away. **Box wins** counts cases where box transpose is faster than that method. Against `.contiguous()` that count is low here precisely because no routing is applied; Section 3.3 reports it under the gate. ITTPD's memory figure comes from its own planner, not from watching the allocator. On these shapes every block fits in fast on-chip memory, so it never asks for anything global.

Table 2: **Three cases, and who wins each.** Both sweeps together, split by the two numbers from Section 2. τ is how long a run has to be before the memory system reads it efficiently, 1,024 values. The first row is where we beat the copy. The last is where our method is the wrong tool.

		Median ms			Box faster than	
Regime	Cases	Box	Copy	Best other	Copy	Best other
$D_{post} \geq \tau$, one small square swap	22	4.03	7.68	8.16	22/22	22/22
$D_{post} \geq \tau$, otherwise	69	12.48	7.69	22.37	11/69	61/69
$D_{post} < \tau$	11	228.12	7.48	74.28	0/11	0/11

Copy is PyTorch `.contiguous()`. **Best other** is whichever in-place method was fastest on that case. Both tests are worked out from the shape before any data is touched. The last row is where our method falls down: a permutation that moves the innermost axis forces a swap with $D_{post} = 1$, each value we move is then a lone number, and the published in-place methods are faster. We report this rather than route around it.

were the best in-place method on 4 of those 4. In the other 11 no back group survived. We were the worst in-place method on every one. We took 146 to 314 ms. The copy took 7 to 14 ms. We were about 30× slower.

We still finish. We still ask for no extra memory. But here the published in-place methods are several times faster than we are. A real system would send these cases to one of them. We have not built that. We report the gap instead of hiding it.

Rank 2 is always in this group. A plain matrix transpose has no back axes to leave alone.

4.2 More than one swap loses to the copy

Section 2.3 shows a single swap always moves at least 0.50 of the tensor. So a route with two or more swaps moves at least as much as a copy does. We are often still the best in-place choice there. But we are not faster than copying. A better search would not change that.

4.3 Planning is slow at high rank

The search runs over orderings of the axes. It stops as soon as it reaches the target. So the common cases, which need one swap, come back at once. But a hard permutation at rank 8 took several seconds to plan.

A route depends only on the shape and the permutation. So we work it out once and reuse it. A much higher rank would need a search that aims at the target instead of spreading out in every direction.

4.4 One GPU

Every time in this paper comes from one NVIDIA GeForce RTX 2070. The main speedup is arithmetic. Equation (1) has no hardware term in it. So it should hold on other machines. The comparisons against other systems are measurements, and those need running again elsewhere.

We should also say this. Our rebuild of ITTPD runs its on-chip path about ten times slower than a plain copy on these shapes. That may be our rebuild rather than the published method. If so, our lead over it is smaller than we report.

5 Earlier Work

5.1 Transposing a matrix in place

Catanzaro, Keller and Garland [1] split a matrix transpose into row and column shuffles, with a rotation in between. Their method needs a temporary buffer. Its cycle-walking step runs one block per cycle. So it can only spread as wide as the number of cycles. They say this themselves.

Gustavson, Karlsson and Kågström [5] work out where each cycle starts using number theory. They do not have to search for it. We use the same idea for square blocks. There a transpose is its own inverse, so the test becomes one comparison.

Sung et al. [11] and Gómez-Luna et al. [4] pad dimensions to sizes that factor well. Padding makes the array bigger. That works against saving memory. In our test the worst case asked

for many times the size of the tensor.

5.2 Permuting more than two dimensions in place

The closest work is ITTPD [2]. It breaks an N-D permutation into block transpositions called $\rho(i, j, k)$. **That is the same operation as our box swap. We do not claim it as ours.**

We differ in two ways.

First, ITTPD runs each step with Catanzaro's two-dimensional method. That needs a temporary buffer of $\max(M, K) \cdot W$. It also goes over the data at least once per step. We walk cycles directly. By Section 2.3 we can go over only part of the data.

Second, ITTPD must choose its breakdown to fit a memory budget. When two choices tie, it picks on memory rather than on time. Our memory cost is tiny whichever breakdown we pick. So we are free to pick the cheapest one.

Where a back group of axes survives, we were faster on 87 of 87 cases. Where it does not, ITTPD is faster than we are. Section 4 has those numbers.

5.3 Copying out of place

The thing we are replacing is PyTorch's own `.contiguous()` [6, 9]. It is fast. It spreads perfectly over a GPU. It pays for that with a second tensor. Work on memory pressure in large-model training [10] and the PyTorch communication document [12] both treat that cost as a real problem, not a theoretical one.

6 Summary

Box transpose fixes a tensor's layout in place. We tested it on 87 sampled shapes at ranks 3 to 10. We also tested it on 15 permutations picked at random. It found a working plan and gave the right answer every time. It used 144 bytes of scratch space. A copy would have used a second tensor of 1,024 MB.

When a permutation leaves a group of axes at the back alone, we are the fastest in-place method we measured. We beat ITTPD, Gomez-Luna and Gustavson on 87, 87 and 79 of their cases.

When the swap is also one small square block, we beat PyTorch's own copy. We are 1.73× faster. The reason is simple. We skip the cells that are already in the right place. A copy cannot.

We work out both tests from the shape, before touching any data. No number in the cost model was measured on a GPU.

When the tests fail, our method is the slowest one we compared. That happens when the permutation moves the innermost axis. We are about 30× slower there. We report it instead of routing around it.

Team and Responsibilities

Kaveen Jayamanna (KJ). The planner and its cost model (`src/_plan.py`): the box-swap graph, the shortest-path search, and the closed-form fixed-point count the search is weighted by. The in-place cycle-following CUDA kernels (`src/_kernels/`), including the vectorised and leader-based paths.

Alexis Newman (AN). Measurement and validation: the correctness suite that gates every result, the sweep over the shape space, the head-to-head against the three published in-place systems, and the adversarial sweep that establishes the method's limitation.

Both members share the write-up and the presentation.

References

[1] Bryan Catanzaro, Alexander Keller, and Michael Garland. A decomposition for in-place matrix transposition. In *Proceedings of the 19th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP '14)*, pages 193–206, Orlando, FL, USA, 2014. ACM. DOI https://doi.org/10.1145/2555243.2555253. Also issued as ACM SIGPLAN Notices 49(8):193–206.

[2] Kai-Jung Cheng and Che-Rung Lee. ITTPD: In-place tensor transposition with permutation decomposition on GPUs. In *Proceedings of the International Conference on High Performance Computing in Asia-Pacific Region (HPCAsia '25)*, pages 90–98, Hsinchu, Taiwan, 2025. DOI https://doi.org/10.1145/3712031.3712033.

[3] David S. Dummit and Richard M. Foote. *Abstract Algebra*. John Wiley & Sons, Hoboken, NJ, 3rd edition, 2003. Section 1.3 gives the decomposition of a permutation into disjoint cycles, unique up to the order of the cycles.

[4] Juan Gómez-Luna, I-Jui Sung, Li-Wen Chang, José María González-Linares, Nicolás Guil, and Wen-mei W. Hwu. In-place matrix transposition on GPUs. *IEEE Transactions on Parallel and Distributed Systems*, 27(3):776–788, 2016. DOI https://doi.org/10.1109/TPDS.2015.2412549.

[5] Fred Gustavson, Lars Karlsson, and Bo Kågström. Parallel and cache-efficient in-place matrix storage format conversion. *ACM Transactions on Mathematical Software*, 38(3):17:1–17:32, 2012. Article 17. DOI https://doi.org/10.1145/2168773.2168775.

[6] Adam Paszke. Add more Tensor methods, June 2016. PyTorch GitHub commit `60f9834`, https://github.com/pytorch/pytorch/commit/60f9834ac6ccb8721548e779d1d3db922d06e6e6, authored 9 June 2016. Registers `contiguous` in the tensor method table of `torch/csrc/generic/Tensor.cpp`.

[7] PyTorch Contributors. `torch.permute`. https://docs.pytorch.org/docs/2.13/generated/torch.permute.html, 2026. PyTorch 2.13 documentation. Accessed August 2026. Archived at https://web.archive.org/web/20260808203715/https://docs.pytorch.org/docs/2.13/generated/torch.permute.html.

[8] PyTorch Contributors. `torch.transpose`. https://docs.pytorch.org/docs/2.13/generated/torch.transpose.html, 2026. PyTorch 2.13 documentation. Accessed August 2026. Archived at https://web.archive.org/web/20260830062342/https://docs.pytorch.org/docs/2.13/generated/torch.transpose.html.

[9] PyTorch Team. PyTorch 2.x: Faster, more pythonic and as dynamic as ever. https://pytorch.org/get-started/pytorch-2-x/, 2026. Accessed August 2026. Archived at https://web.archive.org/web/20260819091658/https://pytorch.org/get-started/pytorch-2-x/.

[10] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. ZeRO: Memory optimizations toward training trillion parameter models. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC20)*, pages 20:1–20:16. IEEE/ACM, 2020. Article 20. DOI https://doi.org/10.1109/SC41405.2020.00024. Also arXiv:1910.02054. The fragmentation claim is supported by its Section 3, "Managing fragmented Memory".

[11] I-Jui Sung, Geng Daniel Liu, and Wen-mei W. Hwu. DL: A data layout transformation system for heterogeneous computing. In *2012 Innovative Parallel Computing (InPar)*, pages 1–11. IEEE, May 2012. DOI https://doi.org/10.1109/InPar.2012.6339606.

[12] Ke Wen. [RFC] communication primitives for non-contiguous tensors. https://github.com/pytorch/pytorch/issues/177427, March 2026. PyTorch GitHub issue #177427, opened 13 March 2026. Source of every RFC quotation in this paper. Accessed August 2026.